

EXPERIMENT 1

% MATLAB Code for Daily Load Curve and Load Duration Curve

%% 1. Input Data

% Time in hours (1 to 24)

time = 1:24;

% Sample hourly load data in kW (Replace with your experiment data)

load_data = [30, 35, 32, 40, 50, 65, 80, 90, 85, 75, 70, 65, ...
60, 65, 70, 85, 100, 110, 95, 80, 70, 55, 45, 35];

% Total connected load in kW (Replace with your system's connected load)
connected_load = 150;

%% 2. Calculations

% Maximum Demand: The highest load value in the 24-hour period
max_demand = max(load_data);

% Average Load: Total energy consumed (area under curve) / 24 hours
avg_load = sum(load_data) / 24;

% Load Factor: Average Load / Maximum Demand
load_factor = avg_load / max_demand;

% Demand Factor: Maximum Demand / Total Connected Load
demand_factor = max_demand / connected_load;

%% 3. Display Results in Command Window

fprintf('\n--- System Load Characteristics ---\n');

fprintf('Maximum Demand: %.2f kW\n', max_demand);

fprintf('Average Load : %.2f kW\n', avg_load);

fprintf('Load Factor : %.4f (%.2f %%)\n', load_factor, load_factor * 100);

fprintf('Demand Factor : %.4f (%.2f %%)\n', demand_factor, demand_factor * 100);

%% 4. Plotting the Curves

figure('Name', 'Load Curves', 'NumberTitle', 'off', 'Position', [100,

% --- Plot 1: Daily Load Curve ---

subplot(2,1,1);

% We use the 'stairs' function because load data is typically represented by discrete values.
stairs([0, time], [load_data(1), load_data], 'b-', 'LineWidth', 1.5);
title('Daily Load Curve', 'FontSize', 12, 'FontWeight', 'bold');

xlabel('Time (Hours)', 'FontSize', 10);

ylabel('Load (kW)', 'FontSize', 10);

grid on;

axis([0 24 0 max_demand*1.2]); % Scales the Y-axis neatly

xticks(0:2:24); % Sets X-axis ticks every 2 hours

% --- Plot 2: Load Duration Curve ---

subplot(2,1,2);

% Sort the load data in descending order for the duration curve

sorted_load = sort(load_data, 'descend');

% We use 'plot' for the duration curve to show the continuous decline
plot(time, sorted_load, 'r-o', 'LineWidth', 1.5, 'MarkerFaceColor', 'r');
title('Load Duration Curve', 'FontSize', 12, 'FontWeight', 'bold');

xlabel('Duration (Hours)', 'FontSize', 10);

ylabel('Load (kW)', 'FontSize', 10);

grid on;

axis([0 24 0 max_demand*1.2]);

xticks(0:2:24);

>> untitled

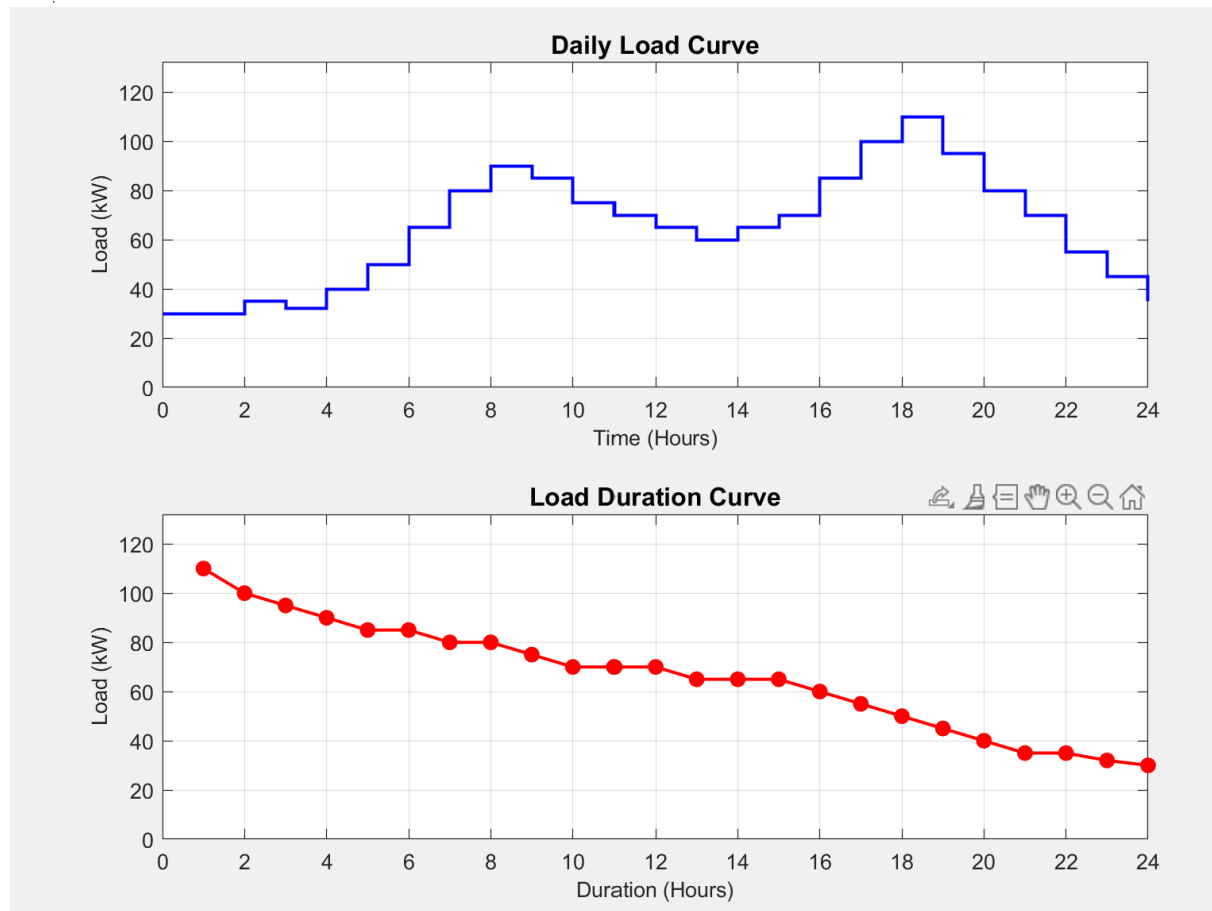
--- System Load Characteristics ---

Maximum Demand: 110.00 kW

Average Load : 66.12 kW

Load Factor : 0.6011 (60.11 %)

Demand Factor : 0.7333 (73.33 %)



EXPERIMENT 2

<pre>% Experiment 2: Future Load Demand Prediction using Linear Regression %% 1. Input Historical Data % Historical Years (X-axis) years = [2018, 2019, 2020, 2021, 2022, 2023, 2024]; % Historical Peak Load Demand in MW (Y-axis) - Replace with your lab data historical_load = [120, 128, 135, 145, 150, 162, 170]; % Future years to predict future_years = [2025, 2026, 2027, 2028, 2030]; %% 2. Linear Regression Calculation % Use polyfit to find the coefficients of a 1st-degree polynomial (Line of Best Fit) % The model is: Load = m * Year + c % 'coefficients(1)' is the slope (m) % 'coefficients(2)' is the y-intercept (c) coefficients = polyfit(years, historical_load, 1); m = coefficients(1); c = coefficients(2); %% 3. Predict Future Demand % Calculate predicted loads for historical years (to draw the trend line) trend_load = polyval(coefficients, years); % Calculate predicted loads for the future years predicted_load = polyval(coefficients, future_years); %% 4. Display Results in Command Window fprintf('\n--- Linear Regression Load Forecasting ---\n'); fprintf('Linear Equation: Load = (%.4f * Year) + (%.4f)\n\n', m, c);</pre>	<pre>fprintf('Predicted Future Loads:\n'); for i = 1:length(future_years) fprintf('Year %d: %.2f MW\n', future_years(i), predicted_load(i)); end fprintf('-----\n\n'); %% 5. Plotting the Results figure('Name', 'Load Forecasting', 'NumberTitle', 'off', 'Position', [100, 100, 600, 400]); % Plot historical data points plot(years, historical_load, 'bo', 'MarkerSize', 8, 'MarkerFaceColor', 'white'); hold on; % Plot the linear regression trend line over historical data plot(years, trend_load, 'k--', 'LineWidth', 1.5, 'DisplayName', 'Regression Trend Line'); % Plot the predicted future data points plot(future_years, predicted_load, 'rs', 'MarkerSize', 8, 'MarkerFaceColor', 'white'); % Draw a line connecting the last historical point to the future predicted point combined_years = [years(end), future_years(1)]; combined_loads = [trend_load(end), predicted_load(1)]; plot(combined_years, combined_loads, 'r-', 'LineWidth', 1.5, 'HandleVisibility', 'off'); % Formatting the plot title('Power System Load Forecasting using Linear Regression'); xlabel('Year', 'FontSize', 10); ylabel('Peak Load Demand (MW)', 'FontSize', 10); legend('Location', 'northwest', 'FontSize', 10); grid on;</pre>
---	---

```
>> untitled2
```

```
--- Linear Regression Load Forecasting ---
Linear Equation: Load = (8.3214 * Year) + (-16673.3214)
```

```
Predicted Future Loads:
```

```
Year 2025: 177.57 MW
```

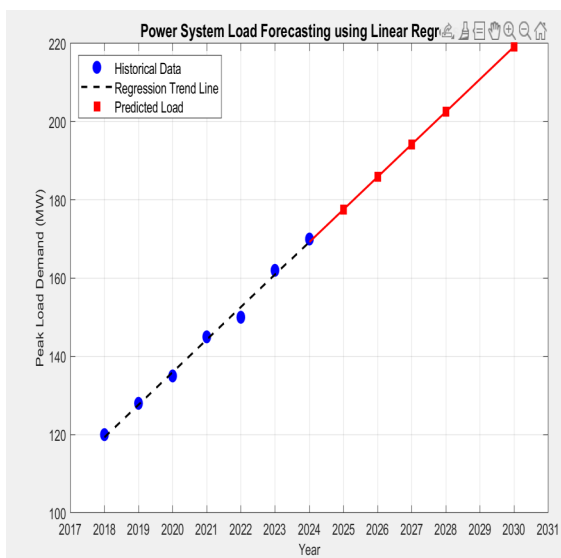
```
Year 2026: 185.89 MW
```

```
Year 2027: 194.21 MW
```

```
Year 2028: 202.54 MW
```

```
Year 2030: 219.18 MW
```

```
-----
```



EXPERIMENT 3

```
% Single Area Load Frequency Control (LFC)
% Frequency Response for Step Load Disturbances
```

```
clc;
clear;
close all;
```

```
% System Parameters
R = 2.4;      % Speed Regulation (Hz/pu MW)
D = 0.8;      % Load Damping Coefficient
H = 5;        % Inertia Constant
f0 = 50;      % Nominal Frequency (Hz)
```

```
% Generator-Load Parameters
Tp = (2*H)/(D*f0);
Kp = 1/D;
```

```
% Transfer Function of Generator-Load Model
s = tf('s');
Gp = Kp/(Tp*s + 1);
```

```
% Governor Transfer Function
Tg = 0.08;
Gg = 1/(Tg*s + 1);
```

```
% Turbine Transfer Function
Tt = 0.3;
Gt = 1/(Tt*s + 1);
```

```
% Open Loop Transfer Function
G = Gg*Gt*Gp;
```

```
% Closed Loop Transfer Function
Sys = feedback(G,1/R);
```

```
% Simulation Time
t = 0:0.01:50;
```

```
% Different Step Load Disturbances
PL1 = 0.01;
PL2 = 0.02;
PL3 = 0.05;
```

```
% Responses
[y1,t1] = step(-PL1*Sys,t);
[y2,t2] = step(-PL2*Sys,t);
[y3,t3] = step(-PL3*Sys,t);
```

```
% Plot Results
figure;
plot(t1,y1,'LineWidth',2);
hold on;
plot(t2,y2,'LineWidth',2);
plot(t3,y3,'LineWidth',2);
grid on;
```

```
xlabel('Time (s)');
ylabel('Frequency Deviation (Hz)');
title('Single Area LFC Response for Different Load Disturbances');
```

```
legend('\DeltaP_L = 0.01 pu',...
'\DeltaP_L = 0.02 pu',...
'\DeltaP_L = 0.05 pu');
```

Load Disturbance 0.01 pu

ans =

struct with fields:

```
RiseTime: 0.5461
TransientTime: 1.4126
SettlingTime: 1.4126
SettlingMin: -0.0085
SettlingMax: -0.0075
Overshoot: 2.9845
Undershoot: 0
Peak: 0.0085
PeakTime: 1.1623
```

Load Disturbance 0.02 pu

ans =

struct with fields:

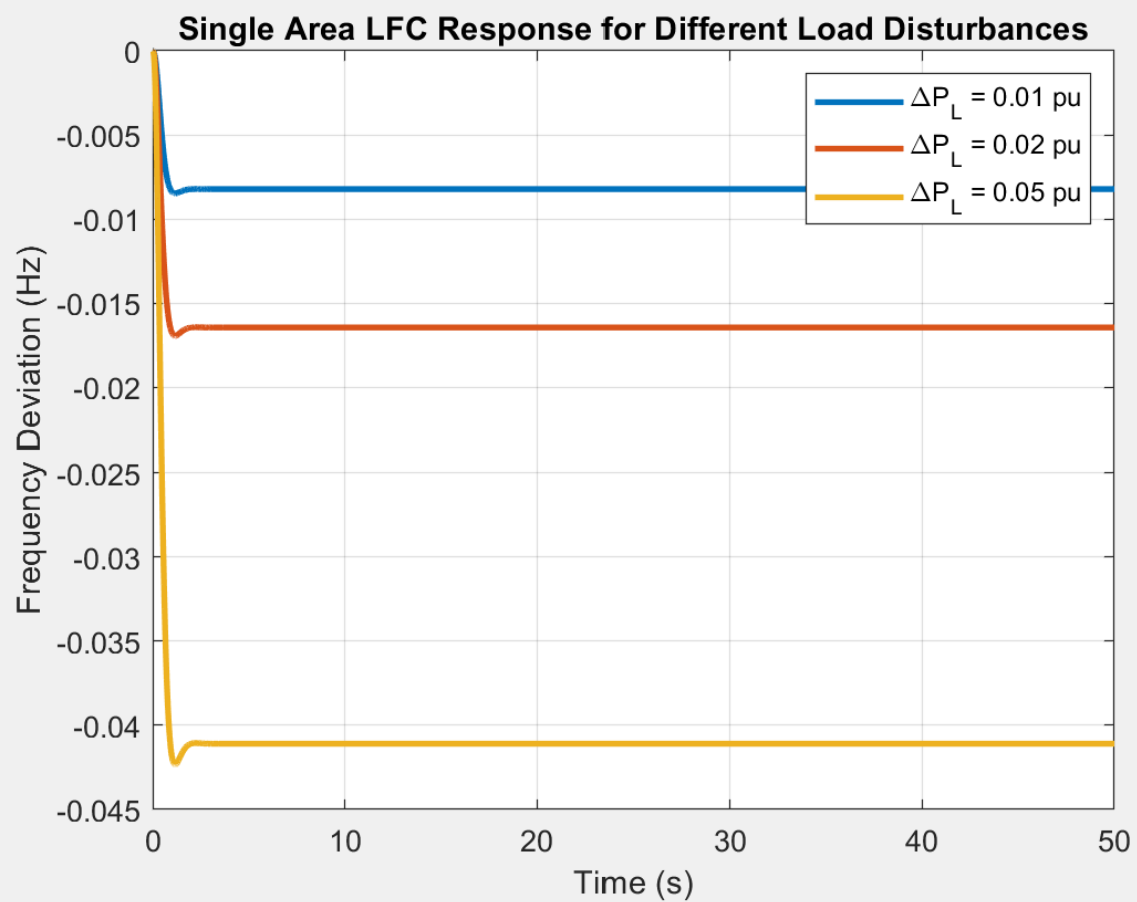
```
RiseTime: 0.5461
TransientTime: 1.4126
SettlingTime: 1.4126
SettlingMin: -0.0169
SettlingMax: -0.0149
Overshoot: 2.9845
Undershoot: 0
Peak: 0.0169
PeakTime: 1.1623
```

Load Disturbance 0.05 pu

ans =

struct with fields:

RiseTime: 0.5461
TransientTime: 1.4126
SettlingTime: 1.4126
SettlingMin: -0.0423
SettlingMax: -0.0373
Overshoot: 2.9845
Undershoot: 0
Peak: 0.0423
PeakTime: 1.1623



EXPERIMENT 4

```
% Speed Governing System of a Synchronous Generator  
% Droop Characteristic Analysis
```

```
clc;  
clear;  
close all;
```

```
% Parameters
```

```
R = 0.05;      % 5% Speed Regulation (Droop)  
Tg = 0.2;      % Governor Time Constant (sec)
```

```
% Transfer Function of Governor
```

```
s = tf('s');  
Ggov = 1/(Tg*s + 1);
```

```
% Step Response of Governor
```

```
figure;  
step(Ggov);  
grid on;  
title('Speed Governor Step Response');
```

```
% Speed Deviation Range
```

```
df = -0.05:0.005:0;
```

```
% Load Change Using Droop Relation
```

```
dP = -(1/R)*df;
```

```
% Plot Droop Characteristic
```

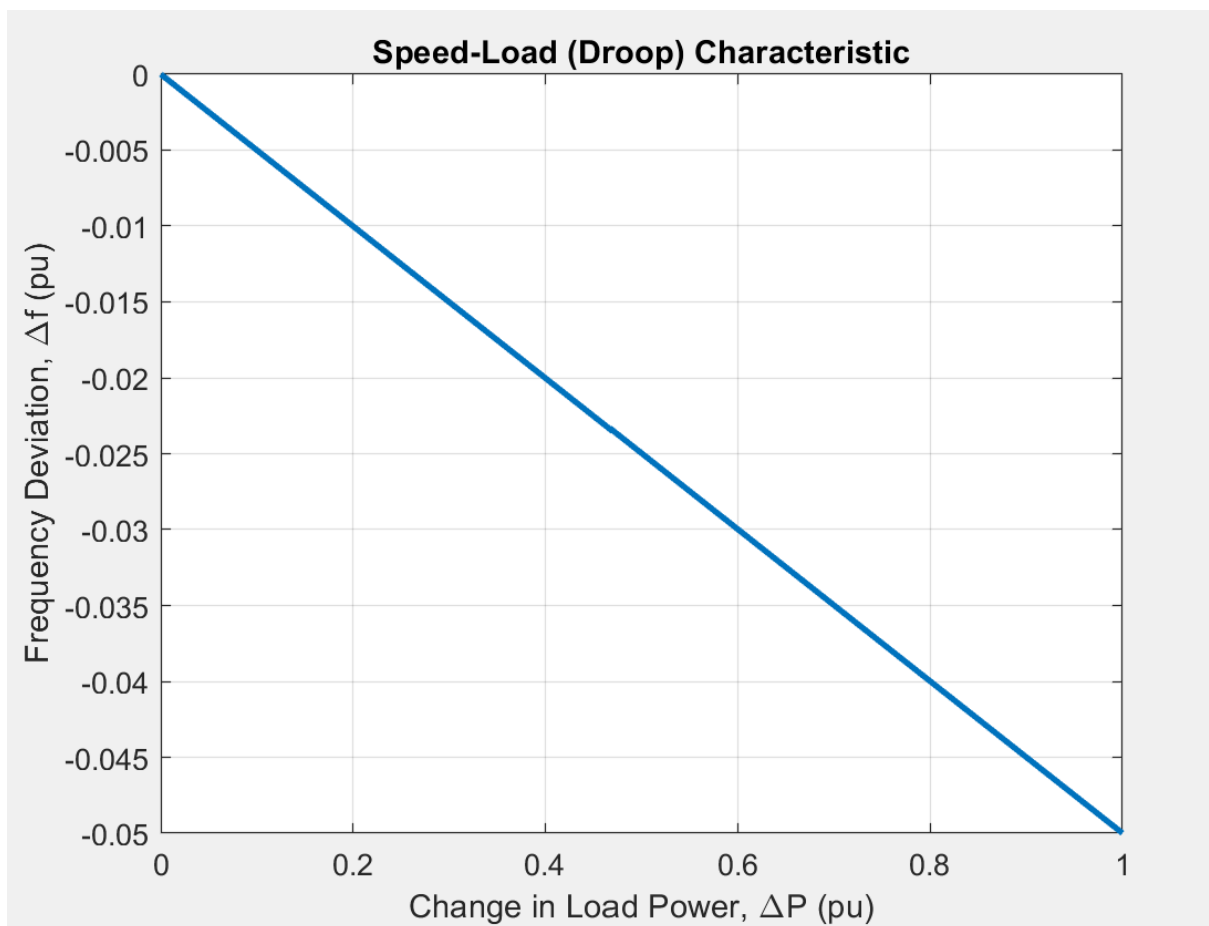
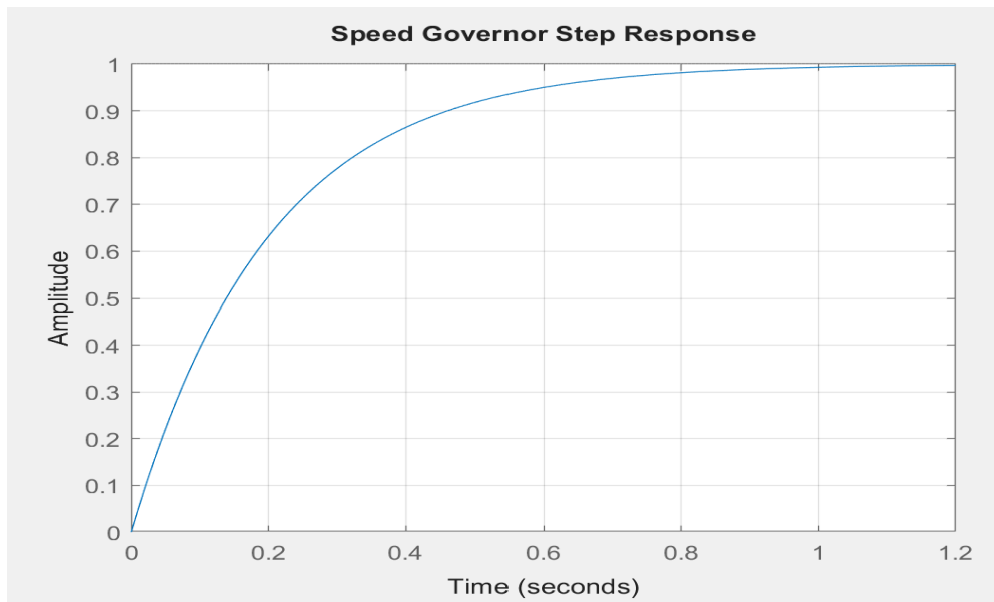
```
figure;  
plot(dP,df,'LineWidth',2);  
grid on;
```

Frequency Deviation	Load Change
---------------------	-------------

-0.0500	1.0000
-0.0450	0.9000
-0.0400	0.8000
-0.0350	0.7000
-0.0300	0.6000
-0.0250	0.5000
-0.0200	0.4000
-0.0150	0.3000
-0.0100	0.2000
-0.0050	0.1000
0	0

Governor Performance

RiseTime: 0.4394
TransientTime: 0.7824
SettlingTime: 0.7824
SettlingMin: 0.9045
SettlingMax: 0.9993
Overshoot: 0
Undershoot: 0
Peak: 0.9993
PeakTime: 1.4644



EXPERIMENT 5

```
% Automatic Generation Control (AGC)
% Single Area Power System

clc;
clear;
close all;

% System Parameters
R = 2.4;          % Speed Regulation
D = 0.8;          % Load Damping Coefficient
H = 5;           % Inertia Constant
f0 = 50;          % Rated Frequency

% Governor Parameters
Tg = 0.08;

% Turbine Parameters
Tt = 0.3;

% Generator-Load Parameters
Tp = (2*H)/(D*f0);
Kp = 1/D;

% Transfer Function Variable
s = tf('s');

% Governor Transfer Function
Gg = 1/(Tg*s + 1);

% Turbine Transfer Function
Gt = 1/(Tt*s + 1);

% Generator-Load Transfer Function
Gp = Kp/(Tp*s + 1);

% Open Loop Transfer Function
G = Gg*Gt*Gp;

% AGC Closed Loop System
Sys = feedback(G,1/R);

% Step Load Disturbance
t = 0:0.01:50;
PL = 0.02;

[y,t] = step(-PL*Sys,t);

% Plot Frequency Response
figure;
plot(t,y,'LineWidth',2);
grid on;

xlabel('Time (s)');
ylabel('Frequency Deviation (Hz)');
title('Single Area AGC Frequency Response');

% Performance Parameters
info = stepinfo(-PL*Sys);

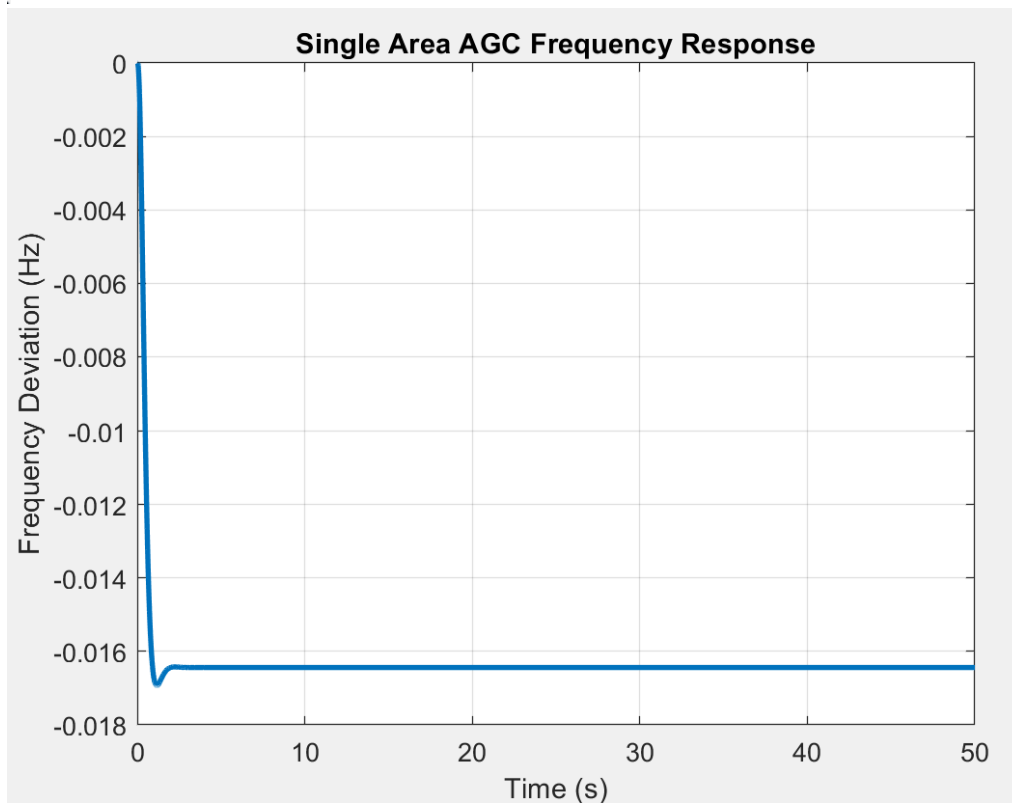
disp('Performance Characteristics');
disp(info);

% Steady State Frequency Deviation
ss_error = dcgain(-PL*Sys);
```


Performance Characteristics

RiseTime: 0.5461
TransientTime: 1.4126
SettlingTime: 1.4126
SettlingMin: -0.0169
SettlingMax: -0.0149
Overshoot: 2.9845
Undershoot: 0
Peak: 0.0169
PeakTime: 1.1623

Steady State Frequency Deviation = -0.016438 Hz



EXPERIMENT 6

```

% Two-Area Load Frequency Control (LFC)
clc;
clear;
close all;

% Area-1 Parameters
R1 = 2.4;
D1 = 0.8;
Tg1 = 0.08;
Tt1 = 0.3;
Kp1 = 120;
Tp1 = 20;

% Area-2 Parameters
R2 = 2.4;
D2 = 0.8;
Tg2 = 0.08;
Tt2 = 0.3;
Kp2 = 120;
Tp2 = 20;

% Tie-Line Synchronizing Coefficient
T12 = 0.545;

s = tf('s');

% Governor Models
Gg1 = 1/(Tg1*s+1);
Gg2 = 1/(Tg2*s+1);

% Turbine Models
Gt1 = 1/(Tt1*s+1);
Gt2 = 1/(Tt2*s+1);

% Generator-Load Models
Gp1 = Kp1/(Tp1*s+1);
Gp2 = Kp2/(Tp2*s+1);

% Area Transfer Functions
G1 = Gg1*Gt1*Gp1;
G2 = Gg2*Gt2*Gp2;

% Closed Loop Systems
Sys1 = feedback(G1,1/R1);
Sys2 = feedback(G2,1/R2);

% Time Vector
t = 0:0.01:100;

% Load Disturbance in Area-1
PL = 0.01;

% Frequency Response
[df1,t1] = step(-PL*Sys1,t);
[df2,t2] = step(-0.5*PL*Sys2,t);

% Tie-Line Power Deviation
dPtie = T12*(df1 - df2);

% Plot Frequency Deviations
figure;
plot(t1,df1,'b','LineWidth',2);
hold on;
plot(t2,df2,'r','LineWidth',2);
grid on;

xlabel('Time (s)');
ylabel('Frequency Deviation (Hz)');
title('Two-Area LFC Frequency Response');
legend('\Deltaf_1','\Deltaf_2');

% Plot Tie-Line Power
figure;
plot(t,dPtie,'k','LineWidth',2);
grid on;

xlabel('Time (s)');
ylabel('\DeltaP_{tie} (pu)');
title('Tie-Line Power Deviation');

% Performance Information
disp('Area-1 Response');
stepinfo(-PL*Sys1)

disp('Area-2 Response');
stepinfo(-0.5*PL*Sys2)
|

```

Area-1 Response

ans =

struct with fields:

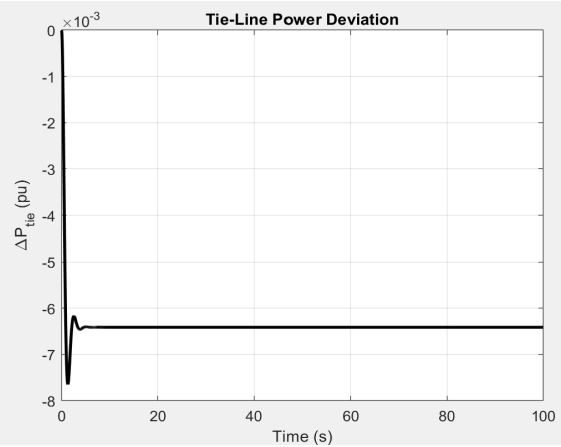
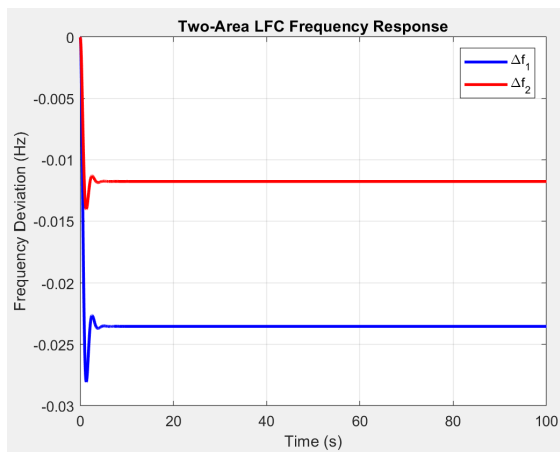
```
RiseTime: 0.5712
TransientTime: 3.0161
SettlingTime: 3.0161
SettlingMin: -0.0281
SettlingMax: -0.0215
Overshoot: 19.2496
Undershoot: 0
Peak: 0.0281
PeakTime: 1.3497
```

Area-2 Response

ans =

struct with fields:

```
RiseTime: 0.5712
TransientTime: 3.0161
SettlingTime: 3.0161
SettlingMin: -0.0140
SettlingMax: -0.0108
Overshoot: 19.2496
Undershoot: 0
Peak: 0.0140
PeakTime: 1.3497
```



EXPERIMENT 7

```
% Experiment 21: Simulation of Automatic Voltage Regulator (AVR) System % Sensor Transfer Function
G_sen = Ks/(1 + Ts*s);

clc;
clear;
close all;

% Amplifier Parameters
Ka = 10;
Ta = 0.1;

% Exciter Parameters
Ke = 1;
Te = 0.4;

% Generator Parameters
Kg = 1;
Tg = 1.0;

% Sensor Parameters
Ks = 1;
Ts = 0.05;

% Transfer Function Variable
s = tf('s');

% Amplifier Transfer Function
G_amp = Ka/(1 + Ta*s);

% Exciter Transfer Function
G_exc = Ke/(1 + Te*s);

% Generator Transfer Function

% Forward Path Transfer Function
G_forward = G_amp * G_exc * G_gen;

% Closed Loop AVR Transfer Function
T_avr = feedback(G_forward, G_sen);

% Time Vector
t = 0:0.01:10;

% Step Response
[y,t] = step(T_avr,t);

% Plot Response
figure;
plot(t,y,'b','LineWidth',2);
hold on;
plot([0 10],[1 1],'r--','LineWidth',1.5);

grid on;
xlabel('Time (sec)');
ylabel('Terminal Voltage V_t (pu)');
title('Step Response of AVR System');
legend('Terminal Voltage', 'Reference Voltage');

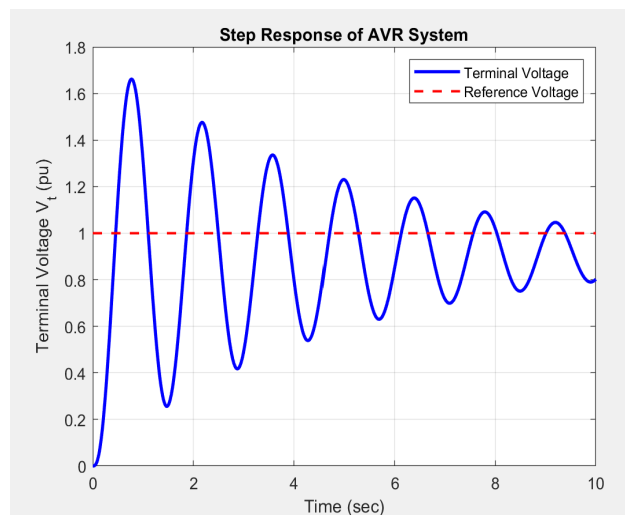
% Performance Parameters
info = stepinfo(T_avr);

disp('Performance Characteristics');
```

Performance Characteristics

```
RiseTime: 0.2534
TransientTime: 19.0812
SettlingTime: 19.0812
SettlingMin: 0.2560
SettlingMax: 1.6618
Overshoot: 82.7933
Undershoot: 0
Peak: 1.6618
PeakTime: 0.7737
```

Steady State Gain = 0.9091



EXPERIMENT 8

```

clc;
clear;
close all;

% Given Values
P = 100;           % Real Power (kW)
PF1 = 0.7;         % Initial Power Factor
PF2 = 0.95;        % Improved Power Factor
V = 400;           % Supply Voltage (V)

% Current Before PF Improvement
I1 = (P*1000)/(sqrt(3)*V*PF1);

% Current After PF Improvement
I2 = (P*1000)/(sqrt(3)*V*PF2);

% Voltage Regulation
VR1 = ((I1-I2)/I1)*100;

% Display Results
fprintf('Current before PF Improvement = %.2f A\n',I1);
fprintf('Current after PF Improvement = %.2f A\n',I2);
fprintf('Percentage Voltage Regulation Improvement = %.2f %%\n',VR1);

% Plot
figure;
bar([I1 I2]);
grid on;
title('Power Factor Improvement');
ylabel('Current (A)');
xlabel('Condition');

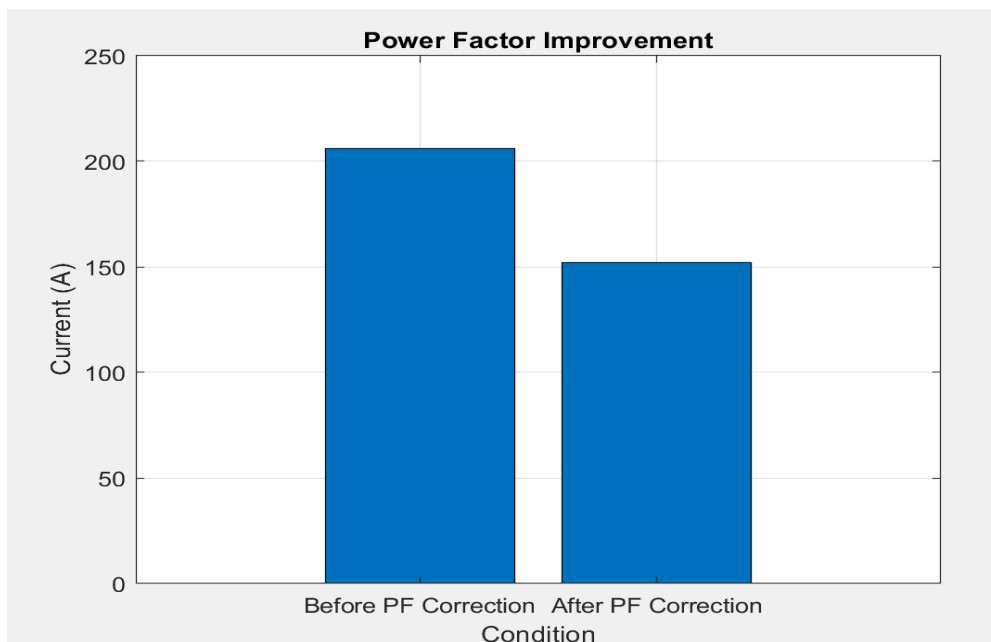
```

Current before PF Improvement = 206.20 A

Current after PF Improvement = 151.93 A

Percentage Voltage Regulation Improvement = 26.32 %

.. |



EXPERIMENT 9

```

clc;
clear;
close all;

% Simulation Time
t = 0:0.1:10;

% Voltage Response without Compensation
V_without = 0.85 + 0.05*(1-exp(-0.2*t));

% Voltage Response with SVC
V_svc = 0.85 + 0.12*(1-exp(-0.5*t));

% Voltage Response with STATCOM
V_statcom = 0.85 + 0.15*(1-exp(-1.0*t));

% Plot Voltage Profiles
figure;
plot(t,V_without,'r','LineWidth',2);
hold on;
plot(t,V_svc,'b','LineWidth',2);
plot(t,V_statcom,'g','LineWidth',2);

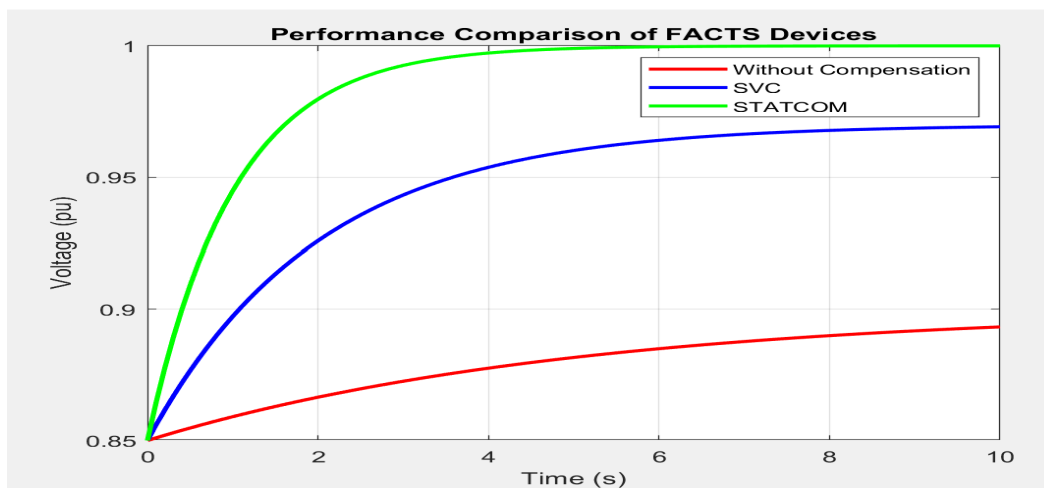
grid on;
xlabel('Time (s)');
ylabel('Voltage (pu)');
title('Performance Comparison of FACTS Devices');
legend('Without Compensation','SVC','STATCOM');

% Reactive Power Support
Q_svc = 80;      % MVAR
Q_statcom = 100; % MVAR

```

Reactive Power by SVC = 80 MVAR

Reactive Power by STATCOM = 100 MVAR



EXPERIMENT 10

```

clc;
clear;
close all;

% Measured Values
z = [1.02; 0.98; 1.01];

% Measurement Matrix
H = [1 0;
     0 1;
     1 1];

% Weight Matrix
W = diag([100 100 50]);

% WLS State Estimation
x = inv(H'*W*H)*(H'*W*z);

% Estimated Measurements
z_est = H*x;

% Residual Error
r = z - z_est;

% Display Results
disp('Estimated State Variables:')
disp(x)

disp('Estimated Measurements:')
disp(z_est)

disp('Residual Errors:')
disp(r)

```

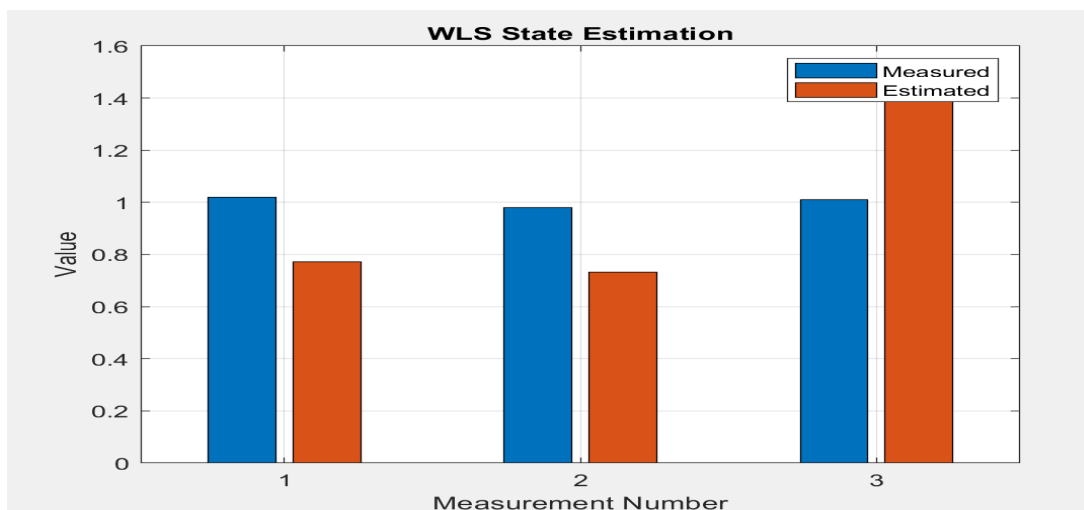
```

Estimated State Variables:
    0.7725
    0.7325

Estimated Measurements:
    0.7725
    0.7325
    1.5050

Residual Errors:
    0.2475
    0.2475
   -0.4950

```



EXPERIMENT 11

```

% SCADA and RTU Simulation

clc;
clear;
close all;

% Input Values
Voltage_Input = 230;    % Voltage in Volts
Current_Input = 10;     % Current in Amperes

% Time Duration
t = 0:1:20;

% RTU Data
Voltage = Voltage_Input * ones(size(t));
Current = Current_Input * ones(size(t));

% Voltage Plot
figure;
subplot(2,1,1);
plot(t,Voltage,'b','LineWidth',2);
grid on;
title('SCADA Voltage Monitoring');
xlabel('Time (s)');
ylabel('Voltage (V)');

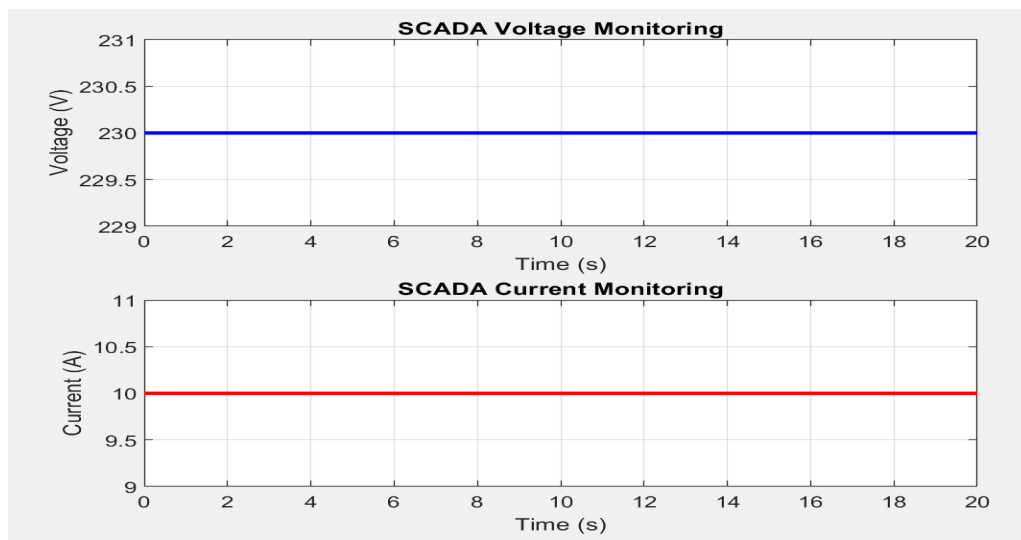
% Current Plot
subplot(2,1,2);
plot(t,Current,'r','LineWidth',2);
grid on;
title('SCADA Current Monitoring');
xlabel('Time (s)');

```

Input Voltage = 230 V

Input Current = 10 A

RTU Data Acquisition Successful



EXPERIMENT 12


```

clc;
clear;
close all;

% Measured Data
z = [1.02; 0.98; 1.01; 1.50];

% Measurement Matrix
H = [1 0;
     0 1;
     1 1;
     1 0];

% Weight Matrix
W = diag([100 100 100 100]);

% WLS State Estimation
x = inv(H'*W*H)*(H'*W*z);

% Estimated Measurements
z_est = H*x;

% Residuals
r = z - z_est;

% Bad Data Threshold
threshold = 0.1;

% Detection
bad_data = abs(r) > threshold;

% Display Results

disp('Estimated States:')
disp(x)

disp('Residuals:')
disp(r)
|
disp('Bad Data Detection:')
disp(bad_data)

% Plot
figure;
bar(abs(r))
grid on
title('Residual Error for Bad Data Detection')
xlabel('Measurement Number')
ylabel('Residual Magnitude')

hold on
yline(threshold, 'r--', 'Threshold')
legend('Residual', 'Threshold')

```

Estimated States:

```

1.0140
0.4880

```

Residuals:

```

0.0060
0.4920
-0.4920
0.4860

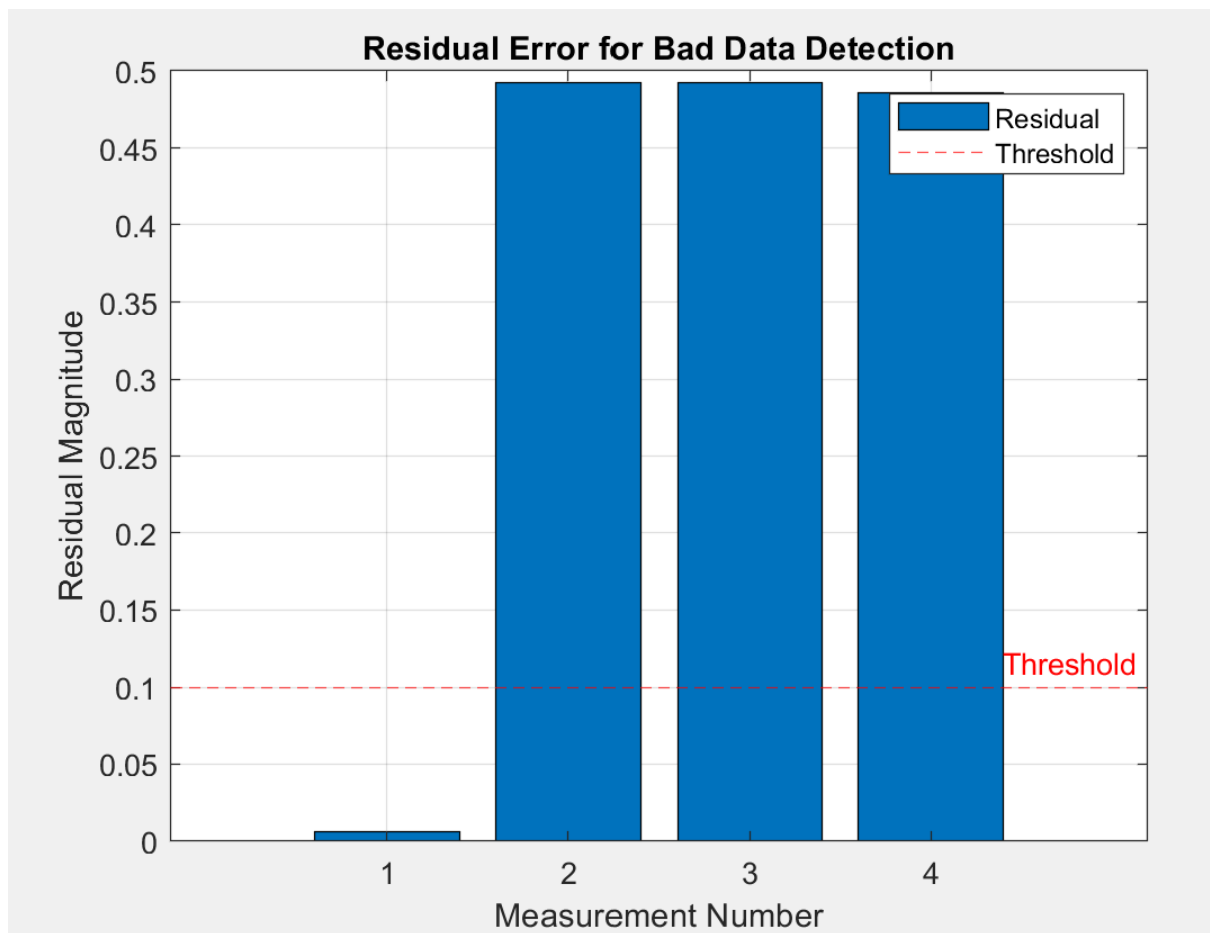
```

Bad Data Detection:

```

0
1
1
1

```



EXPERIMENT 13

```

clc;
clear;
close all;

% Generator Cost Coefficients
a = [500 400]; % Fixed Cost
b = [5.3 5.5]; % Linear Cost Coefficient
c = [0.004 0.006]; % Quadratic Cost Coefficient

% Total Load Demand
PD = 250; % MW

% Lambda Iteration Method
lambda = 7;
tolerance = 0.001;

while true

    P1 = (lambda - b(1))/(2*c(1));
    P2 = (lambda - b(2))/(2*c(2));

    Ptotal = P1 + P2;

    error = PD - Ptotal;

    if abs(error) < tolerance
        break;
    end

    lambda = lambda + 0.001*error;

end

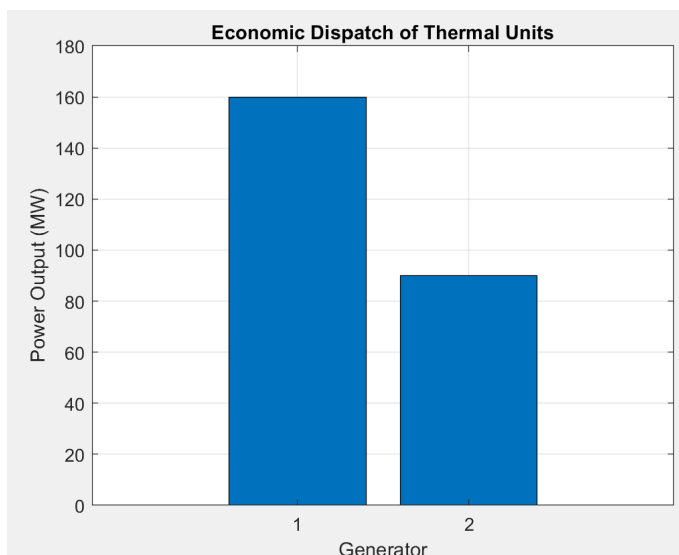
% Total Generation Cost
Cost = a(1)+b(1)*P1+c(1)*P1^2 + ...
       a(2)+b(2)*P2+c(2)*P2^2;

fprintf('Generator 1 Output = %.2f MW\n',P1);
fprintf('Generator 2 Output = %.2f MW\n',P2);
fprintf('Total Generation = %.2f MW\n',P1+P2);
fprintf('Optimal Cost = %.2f Rs/hr\n',Cost);

% Plot
figure;
bar([P1 P2]);
grid on;
title('Economic Dispatch of Thermal Units');
xlabel('Generator');
ylabel('Power Output (MW)');

```

Generator 1 Output = 160.00 MW
 Generator 2 Output = 90.00 MW
 Total Generation = 250.00 MW
 Optimal Cost = 2394.01 Rs/hr



EXPERIMENT 14

```

clc;
clear;
close all;

% Number of Units
n = 3;

% Operating Cost (Rs/hr)
cost = [500 700 900];

% Power Generation Capacity (MW)
P = [100 150 200];

% Load Demand (MW)
PD = 250;

% Initialize
min_cost = inf;
best_combination = [];

% Forward Dynamic Programming
for u1 = 0:1
    for u2 = 0:1
        for u3 = 0:1
            units = [u1 u2 u3];

            total_power = sum(units .* P);
            total_cost = sum(units .* cost);

            if total_power >= PD
                if total_cost < min_cost
                    min_cost = total_cost;
                    best_combination = units;
                end
            end
        end
    end
end

% Display Results
disp('Optimal Unit Commitment:')
disp(best_combination)

fprintf('Total Available Power = %.0f MW\n', ...
        sum(best_combination .* P));

fprintf('Minimum Operating Cost = %.2f Rs/hr\n', ...
        min_cost);

% Plot
figure;
bar(best_combination);
grid on;
title('Optimal Unit Commitment');
xlabel('Generating Units');
ylabel('Status (1=ON, 0=OFF)');

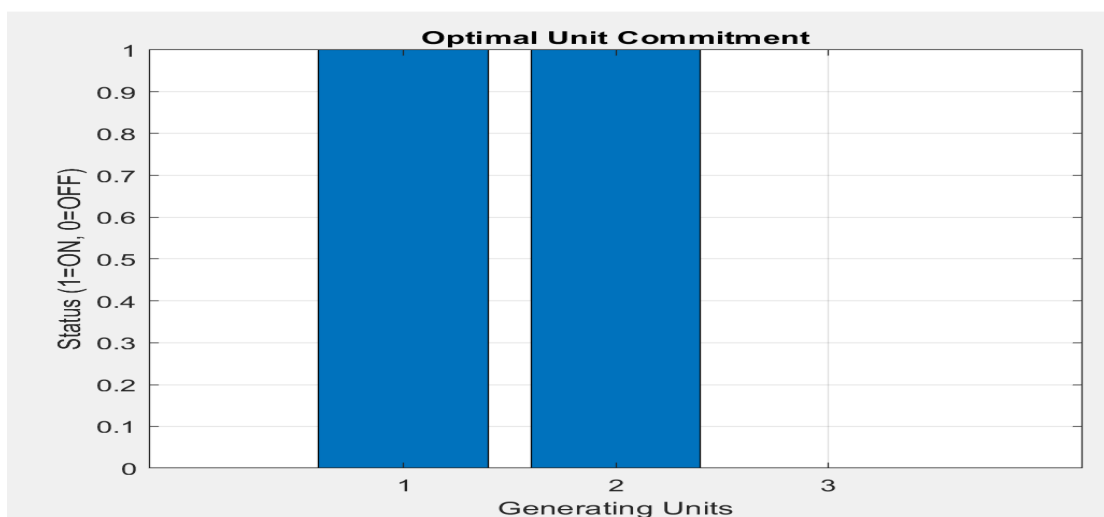
```

Optimal Unit Commitment:

1 1 0

Total Available Power = 250 MW

Minimum Operating Cost = 1200.00 Rs/hr



EXPERIMENT 15

```

clc;
clear;
close all;

% Hourly Load Demand (MW)
Load = [120 110 100 95 90 100 150 200 250 300 320 350 ...
        360 340 320 300 280 350 400 450 420 300 200 150];

Hours = 1:24;

% Load Shifting (Reduce Peak by 50 MW)
Shift_Load = Load;

peak_hour = 20;      % Peak Hour
shift_hour = 4;      % Off-Peak Hour

Shift_Load(peak_hour) = Shift_Load(peak_hour) - 50;
Shift_Load(shift_hour) = Shift_Load(shift_hour) + 50;

% Plot
figure;
plot(Hours, Load, 'r', 'LineWidth', 2);
hold on;
plot(Hours, Shift_Load, 'b', 'LineWidth', 2);
grid on;

xlabel('Hour');
ylabel('Load (MW)');
title('DSM and Load Shifting Analysis');
legend('Before DSM', 'After DSM');

% Peak Load Comparison

```

Peak Load Before DSM = 450 MW

Peak Load After DSM = 420 MW

|

